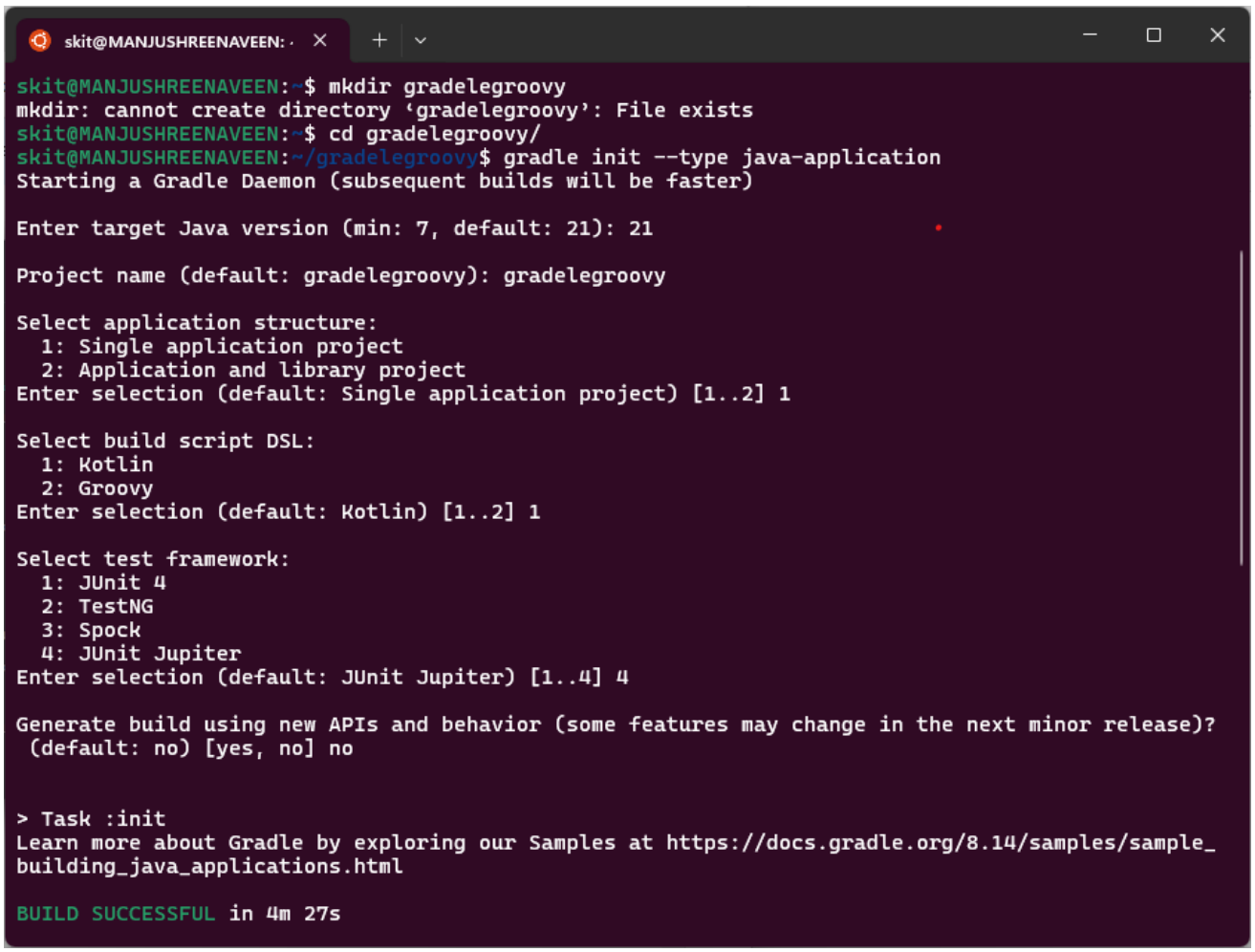
Experiment 3 - Working with Gradle

Step 1: Set Up Gradle Project with Groovy DSL

1. Create and Initialize the Project

```
cd ~
mkdir GradleGroovy
cd GradleGroovy
gradle init --type java-application --dsl groovy --overwrite
```

Choose defaults by pressing Enter for each prompt.



2. Verify Project Structure

The `gradle init` command creates a standard project layout. To view this structure in your terminal, you can use the `tree` command.

Run the following command in your project's root directory (`HelloGradleGroovy`):

```
tree
```

```
skit@MANJUSHREENAVEEN:

BUILD SUCCESSFUL in 4m 27s
1 actionable task: 1 executed
skit@MANJUSHREENAVEEN:~/gradelegroovy$ tree
.
├── app
│   ├── build.gradle.kts
│   └── src
│       ├── main
│       │   ├── java
│       │   │   └── org
│       │   │       └── example
│       │   │           └── App.java
│       │   └── resources
│       └── test
│           ├── java
│           │   └── org
│           │       └── example
│           │           └── AppTest.java
│           └── resources
├── gradle
│   ├── libs.versions.toml
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradle.properties
├── gradlew
├── gradlew.bat
└── settings.gradle.kts

15 directories, 10 files
skit@MANJUSHREENAVEEN:~/gradelegroovy$
```

3. Update `app/build.gradle`

Open the file:

```
gedit app/build.gradle
```

Replace its contents with:

```
plugins {
    // Apply the Java plugin for compiling Java code
    id 'java'
    // Apply the application plugin to add support for building an
application
    id 'application'
}

group = 'org.example'
```

```
version = '1.0'

repositories {
    // Use Maven Central for resolving dependencies.
    mavenCentral()
}

dependencies {
    // Define your dependencies. Use JUnit Jupiter (JUnit 5) for testing.
    testImplementation 'org.junit.jupiter:junit-jupiter-api:5.10.0'
    testRuntimeOnly 'org.junit.jupiter:junit-jupiter-engine:5.10.0'
    // If using parameterized tests
    testImplementation 'org.junit.jupiter:junit-jupiter-params:5.10.0'
}

application {
    // Define the main class for the application.
    mainClass = 'org.example.App'
}

// A custom task example: printing a greeting
task hello {
    doLast {
        println 'Hello, Gradle!'
    }
}
```

4. Dependency Management (For Information Only – Can Be Skipped)

Dependencies are declared in the `dependencies` block. Gradle resolves them from the repositories configured above, like Maven Central.

Example:

```
dependencies{
    testImplementation 'org.junit.jupiter:junit-jupiter-api:5.10.0'
    // Example: Adding a test dependency
}
```

Gradle will download these automatically during the build process.

5. Running and Verifying Your Gradle Project

- **Build the Project:** Compile, test, and package your application:
- `gradle build`
- **Run the Application:** Execute the main class:
- `gradle run`
- **Execute a Custom Task:** Run your custom task by name:
- `gradle hello`

```
skit@MANJUSHREENAVEEN:~/gradelegroovy$ gedit app/build.gradle
skit@MANJUSHREENAVEEN:~/gradelegroovy$ gradle build
Calculating task graph as no cached configuration is available for tasks: build

[Incubating] Problems report is available at: file:///home/skit/gradelegroovy/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 1m 16s
7 actionable tasks: 7 executed
Configuration cache entry stored.
skit@MANJUSHREENAVEEN:~/gradelegroovy$ gradle run
Calculating task graph as no cached configuration is available for tasks: run

> Task :app:run
Hello World!

BUILD SUCCESSFUL in 2s
2 actionable tasks: 1 executed, 1 up-to-date
Configuration cache entry stored.
skit@MANJUSHREENAVEEN:~/gradelegroovy$ gradle hello
Calculating task graph as no cached configuration is available for tasks: hello

> Task :app:hello
Hello, Gradle!

BUILD SUCCESSFUL in 1s
1 actionable task: 1 executed
Configuration cache entry stored.
skit@MANJUSHREENAVEEN:~/gradelegroovy$
```

Step 2: Set Up Gradle Project with Kotlin DSL

1. Create and Initialize a new Project in a new directory

```
cd ~
mkdir HelloGradleKotlin
cd HelloGradleKotlin
gradle init --type java-application --dsl kotlin --overwrite
```

Choose defaults by pressing Enter for each prompt.

```
skit@MANJUSHREENAVEEN:

BUILD SUCCESSFUL in 1s
1 actionable task: 1 executed
Configuration cache entry stored.
skit@MANJUSHREENAVEEN:~/gradelegroovy$ cd ~
mkdir HelloGradleKotlin
cd HelloGradleKotlin
gradle init --type java-application --dsl kotlin --overwrite

Enter target Java version (min: 7, default: 21):

Project name (default: HelloGradleKotlin):

Select application structure:
  1: Single application project
  2: Application and library project
Enter selection (default: Single application project) [1..2]

Select test framework:
  1: JUnit 4
  2: TestNG
  3: Spock
  4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4]

Generate build using new APIs and behavior (some features may change in the next minor release)?
 (default: no) [yes, no]


> Task :init
Learn more about Gradle by exploring our Samples at https://docs.gradle.org/8.14/samples/sample_
building_java_applications.html

BUILD SUCCESSFUL in 42s
1 actionable task: 1 executed
skit@MANJUSHREENAVEEN:~/HelloGradleKotlin$
```

2. Verify Project Structure

The `gradle init` command creates a standard project layout. To view this structure in your terminal, you can use the `tree` command.

Run the following command in your project's root directory (`HelloGradleKotlin`):

```
Tree
```

```
skit@MANJUSHREENAVEEN:

(default: no) [yes, no]


> Task :init
Learn more about Gradle by exploring our Samples at https://docs.gradle.org/8.14/samples/sample_
building_java_applications.html

BUILD SUCCESSFUL in 42s
1 actionable task: 1 executed
skit@MANJUSHREENAVEEN:~/HelloGradleKotlin$ tree
.
├── app
│   ├── build.gradle.kts
│   └── src
│       ├── main
│       │   ├── java
│       │   │   └── org
│       │   │       └── example
│       │   │           └── App.java
│       │   └── resources
│       └── test
│           ├── java
│           │   └── org
│           │       └── example
│           │           └── AppTest.java
│           └── resources
├── gradle
│   ├── libs.versions.toml
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradle.properties
├── gradlew
├── gradlew.bat
└── settings.gradle.kts

15 directories, 10 files
skit@MANJUSHREENAVEEN:~/HelloGradleKotlin$
```

3. Update `app/build.gradle.kts`

Open the file:

```
gedit app/build.gradle.kts
```

Replace its contents with:

```
plugins {
    // Apply the Java plugin for compiling Java code
    java
    // Apply the application plugin to add support for building an
application
    application
}

group = "org.example"
version = "1.0"

repositories {
    mavenCentral() // Configures Maven Central for dependencies
}
```

```
dependencies {
    // Define dependencies using Kotlin DSL syntax. Use JUnit Jupiter
(JUnit 5) for testing.
    testImplementation("org.junit.jupiter:junit-jupiter-api:5.10.0")
    testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.10.0")
    // If using parameterized tests
    testImplementation("org.junit.jupiter:junit-jupiter-params:5.10.0")
}

application {
    mainClass.set("org.example.App") // Specifies the main class
}

// A custom task example using Kotlin DSL
tasks.register("hello") {
    doLast {
        println("Hello, Gradle with Kotlin DSL!")
    }
}
```

4. Dependency Management (For Information Only – Can Be Skipped)

Dependencies are declared in the `dependencies` block. Gradle resolves them from the repositories configured above, like Maven Central.

Example:

```
dependencies {
    // Example: Adding JUnit Jupiter (JUnit 5) test dependencies
    testImplementation("org.junit.jupiter:junit-jupiter-api:5.10.0")
}
```

Gradle will download these automatically during the build process.

5. Running and Verifying Your Gradle Project

- **Build the Project:** Compile, test, and package your application:
- `gradle build`
- **Run the Application:** Execute the main class:
- `gradle run`
- **Execute a Custom Task:** Run your custom task by name:

```
skit@MANJUSHREENAVEEN:
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradle.properties
├── gradlew
├── gradlew.bat
└── settings.gradle.kts

15 directories, 10 files
skit@MANJUSHREENAVEEN:~/HelloGradleKotlin$ gedit app/build.gradle.kts
skit@MANJUSHREENAVEEN:~/HelloGradleKotlin$ gradle build
Calculating task graph as no cached configuration is available for tasks: build

[Incubating] Problems report is available at: file:///home/skit/HelloGradleKotlin/build/reports/
problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if th
ey come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14/userguide/command_line_interface.
html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 7s
7 actionable tasks: 7 executed
Configuration cache entry stored.
skit@MANJUSHREENAVEEN:~/HelloGradleKotlin$ gradle run
Calculating task graph as no cached configuration is available for tasks: run

> Task :app:run
Hello World!

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
Configuration cache entry stored.
skit@MANJUSHREENAVEEN:~/HelloGradleKotlin$
```

Run the Custom Task:

- **Command:**

  gradle hello